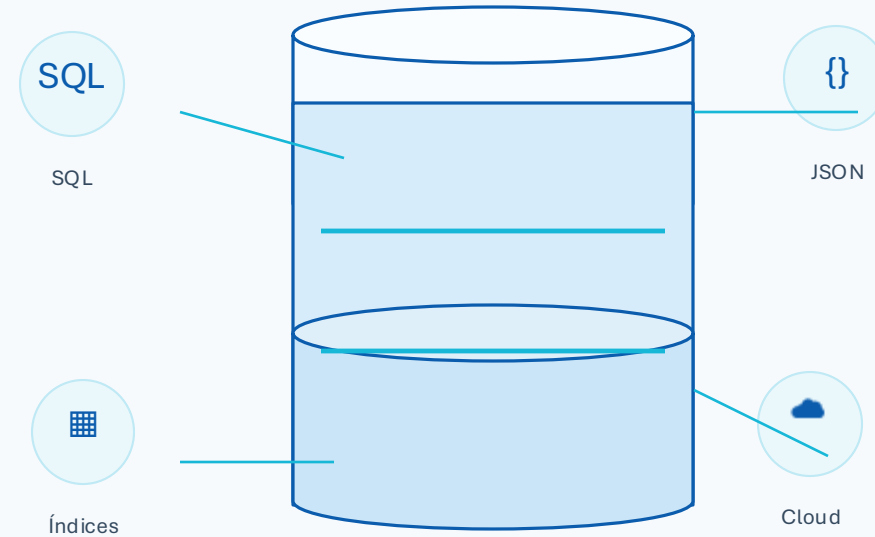


Sistemas de Gestión de Bases de Datos

Unidad 1 · Arquitectura de motores de bases de datos

De la arquitectura conceptual al funcionamiento real de un DBMS
Enfoque multiplataforma · 2026



Objetivo de la unidad

Qué debería poder explicar un estudiante al finalizar la clase

Objetivo de aprendizaje

Comprender cómo se organiza internamente un motor de base de datos y comparar cómo distintos DBMS resuelven problemas comunes: procesamiento de consultas, memoria, almacenamiento, transacciones, metadatos y recuperación.



Procesar

De SQL a plan de ejecución.



Persistir

Datos, índices y archivos físicos.



Recuperar

Logs, checkpoints y transacciones.

La idea central: distintos motores, mismos problemas de ingeniería.

Pregunta disparadora

Para activar saberes previos y orientar la investigación

¿Qué ocurre dentro del motor cuando una aplicación ejecuta una consulta SQL?

Ejemplo: `SELECT saldo FROM cuentas WHERE nro_cuenta = 12345;`

1

¿Dónde está la definición de cuentas?

2

¿Cómo sabe si hay un índice útil?

3

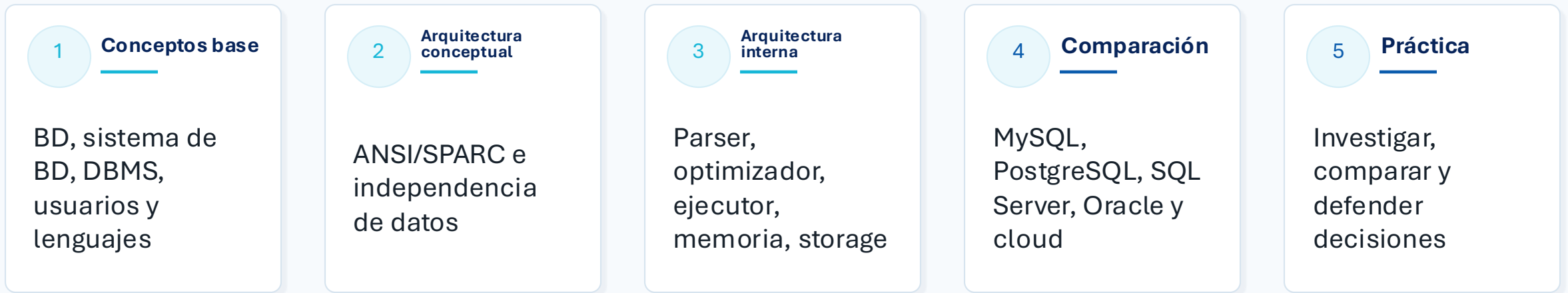
¿Dónde se leen los datos: memoria o disco?

4

¿Qué pasa si el servidor cae?

Mapa de la clase

Teoría breve + investigación aplicada



Eje didáctico: menos memorización de comandos, más comprensión de patrones comunes entre motores.

Evolución: de archivos a plataformas de datos

La arquitectura responde a necesidades históricas concretas



Archivos y cintas

Procesamiento secuencial,
dependencia fuerte de programas.



DBMS relacionales

Modelo relacional, SQL, integridad,
concurrency y transacciones.



Analítica y DW

Columnaridad, OLAP, cargas
masivas y optimización de lectura.



NoSQL y cloud

Escalabilidad, flexibilidad,
operación administrada y costo
variable.

Hoy un ingeniero no solo diseña tablas: selecciona arquitecturas, opera servicios y evalúa costos, seguridad y disponibilidad.

Panorama tecnológico actual

Relacional, NoSQL, nube y automatización conviven



Relacional

Transacciones, integridad y SQL



Documental

JSON y flexibilidad de esquema



Columnar / Analítico

Lectura masiva y compresión



Grafo

Relaciones complejas y recorridos



Cloud DBaaS

Servicio administrado y elasticidad



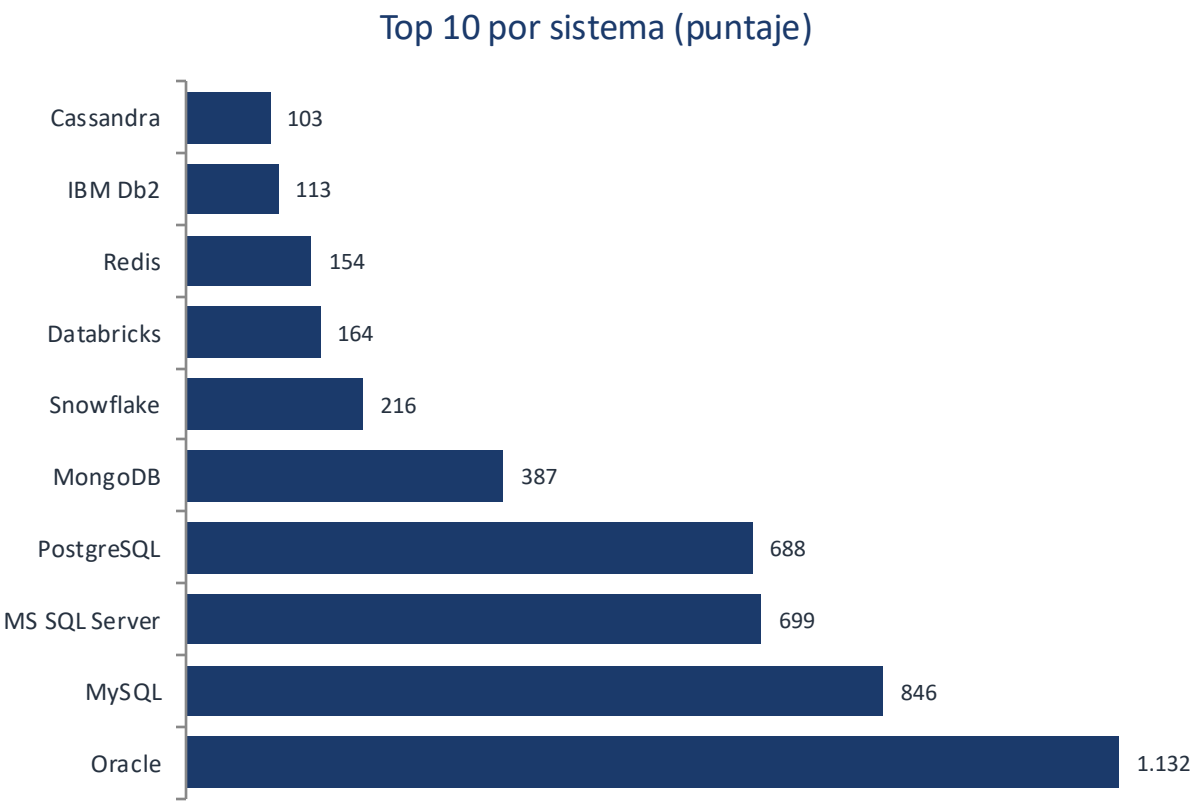
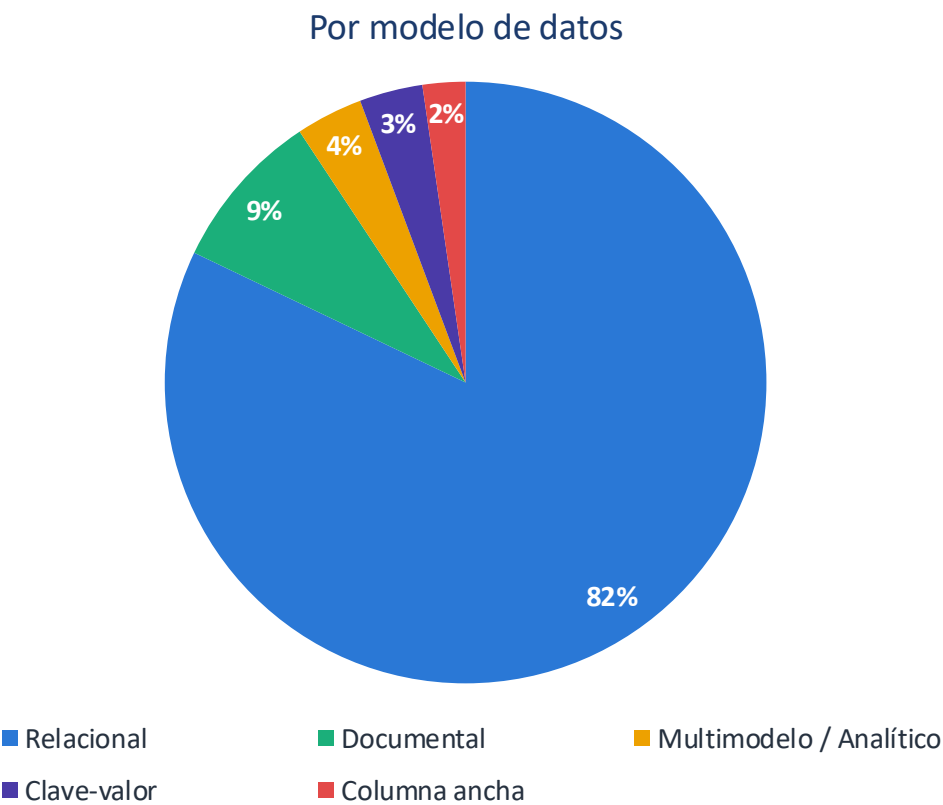
IA / automatización

Asistencia en operación y desarrollo

La selección de motor depende del caso de uso, no de la moda tecnológica.

Panorama de uso: ranking DB-Engines

Popularidad por modelo de datos entre los 10 sistemas más populares (julio 2026)

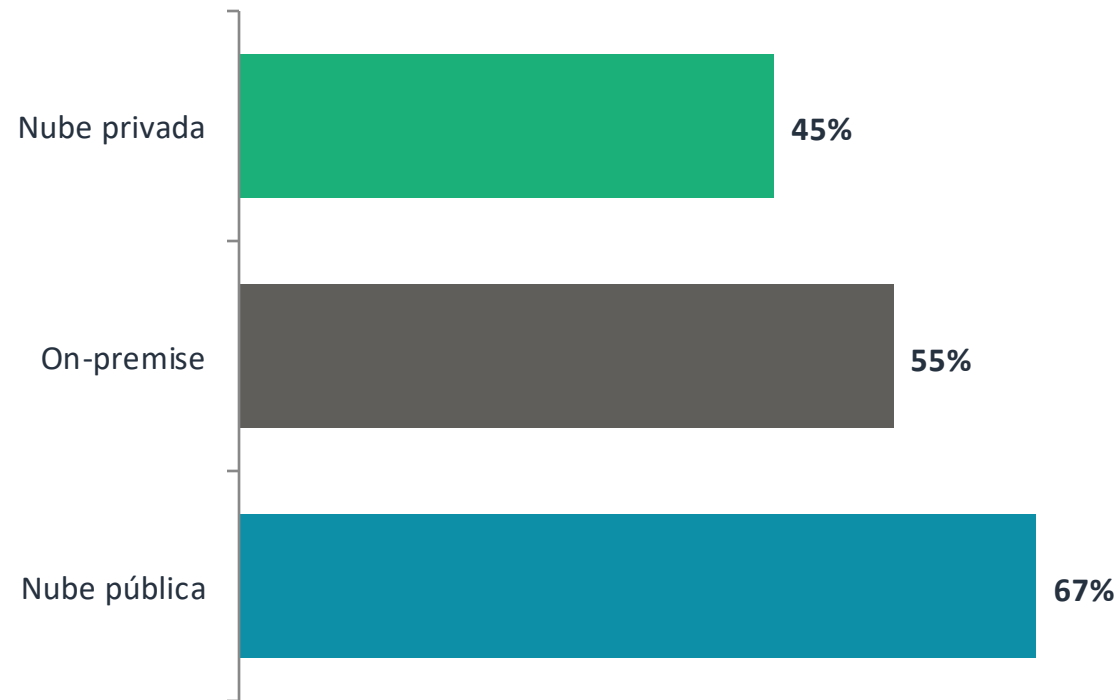


Lectura: el modelo relacional concentra el **82%** de la popularidad del top 10 — la elección del motor sigue el caso de uso, no la moda.

Fuente: DB-Engines Ranking (db-engines.com), julio 2026. La popularidad se mide por búsquedas, ofertas de empleo, menciones técnicas y en redes; no equivale a cuota de mercado ni a cantidad de instalaciones.

On-premise vs. nube: modelos que conviven

Porcentaje de organizaciones que usan cada modelo de despliegue (2026)



Híbrido es la norma

Los porcentajes suman más de 100%: la mayoría combina varios modelos a la vez.

La nube sigue creciendo

El gasto en infraestructura cloud creció ~24% en 2025, impulsado por cargas de IA.

On-premise resurge

Cumplimiento, residencia de datos y costo predecible reavivan lo on-premise en 2026.

Lectura: no es on-premise o nube, sino cómo combinarlos según caso de uso, cumplimiento y costo.

Conceptos base

Tres términos que no deben confundirse

DB

Base de datos

Conjunto persistente y organizado de datos usados por una organización o sistema con un fin determinado. Concepto lógico.

SYS

Sistema de BD

Sistema computarizado que da soporte a una o más bases de datos: hardware, software, datos, usuarios y procesos.

**DBMS / SGBD**

Software que gestiona definición, manipulación, seguridad, concurrencia, transacciones, recuperación y acceso eficiente.

Un DBMS no es solo “guardar datos”: también decide cómo protegerlos, recuperarlos y accederlos eficientemente.

DBMS vs sistemas de archivos

El valor agregado del motor

Sistemas de archivos

El programa suele conocer la estructura exacta de los archivos. La integridad, seguridad, concurrencia y recuperación quedan en gran parte del lado de la aplicación.

DBMS

Centraliza reglas, transacciones, seguridad, metadatos, concurrencia, optimización, backup/restore y acceso compartido a los datos.

- Menor redundancia e inconsistencia
- Mayor integridad y seguridad
- Mejor concurrencia y recuperación
- Independencia entre datos y aplicaciones

Componentes de un sistema de base de datos

La arquitectura no es solo software



Datos

Integrados, persistentes y compartidos



Hardware

CPU, memoria, discos, red y almacenamiento



Software

DBMS, utilidades, drivers, herramientas



Usuarios

DBA, desarrolladores, analistas, usuarios finales

En entornos cloud, parte del hardware y de la operación pasa a ser responsabilidad del proveedor, pero las decisiones técnicas no desaparecen.

Usuarios y roles alrededor del DBMS

Cada perfil mira la base desde una necesidad distinta



DA

Define políticas de datos, gobierno y decisiones estratégicas.



DBA

Administra operación, seguridad, rendimiento, backups y disponibilidad.



Dev

Construye aplicaciones, consultas, APIs y migraciones.



Analista

Consume, interpreta y comunica información para decisiones.

La catedra se enfoca en la mirada del DBA/ingeniero: cómo funciona y cómo se opera el motor.

Lenguajes y operaciones de base de datos

SQL integra varias familias de comandos...

DDL

CREATE, ALTER, DROP
Define objetos y estructuras.

DML

SELECT, INSERT, UPDATE, DELETE
Manipula y consulta datos.

DCL

GRANT, REVOKE
Administra permisos.

TCL

COMMIT, ROLLBACK
Controla transacciones.

Detrás de cada sentencia SQL hay procesamiento, metadatos, optimización, memoria, almacenamiento y control transaccional.

Comprensión Práctica de la Arquitectura ANSI/SPARC

VISTAS

Nivel externo

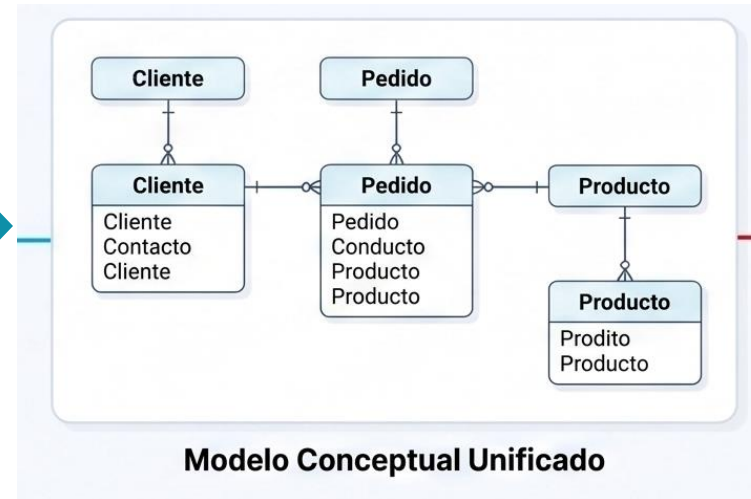
Cómo cada usuario o aplicación ve los datos.



LÓGICA

Nivel conceptual

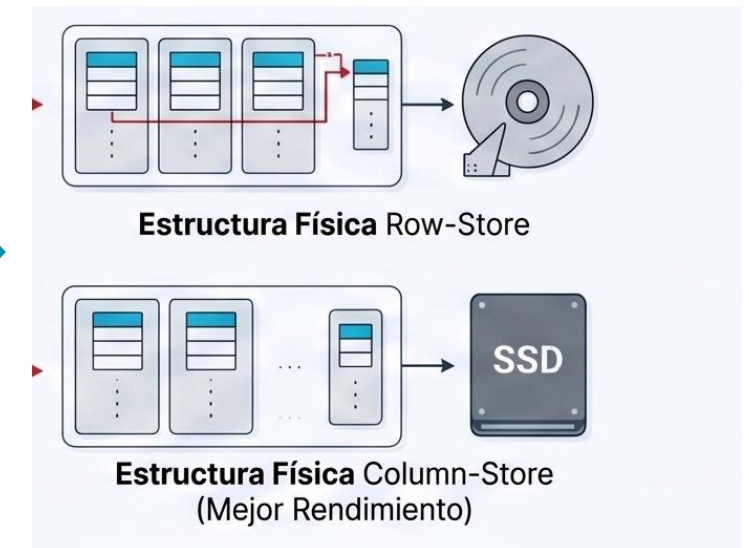
Modelo lógico común de la organización.



ALMACENAMIENTO

Nivel interno

Cómo se almacenan físicamente los datos.



La separación permite modificar una vista o una estructura física sin reescribir toda la aplicación.

Ejemplo: migración de disco (Row-Store) a SSD (Column-Store) sin que los Usuarios A, B o C lo noten, gracias al Nivel Conceptual.

Beneficios Clave

- Independencia de Datos: vistas separadas de lo físico
- Flexibilidad: cambios en lo físico o vistas no afectan a la aplicación
- Modularidad: mayor facilidad de desarrollo

Niveles de abstracción

La misma base puede tener varias miradas

Externo

Vistas parciales para usuarios o aplicaciones.
Ejemplo: una vista que oculta columnas sensibles o simplifica una consulta compleja.

Conceptual

Estructura lógica compartida: entidades, tablas, relaciones, restricciones, dominios y reglas de negocio.

Interno

Organización física: páginas, bloques, índices, archivos, particiones, tablaspace, logs y estructuras de acceso.

Para un DBA, el nivel interno es clave; para el usuario final, debería ser invisible.

Independencia de datos

Cambiar sin romper

L

Independencia lógica

Modificar el esquema conceptual sin afectar necesariamente las vistas externas. Ejemplo: agregar atributos o dividir una entidad preservando vistas.

F

Independencia física

Modificar estructuras internas sin cambiar la visión lógica. Ejemplo: crear índices, cambiar tablespaces o particionar tablas.

El ideal de un DBMS: que una decisión física de rendimiento no obligue a reprogramar el sistema.

Modelos de datos

El modelo condiciona cómo se diseña, consulta y escala

Relacional

Tablas, claves, restricciones y SQL.

Documental

Documentos JSON/BSON y agregados.

Clave-valor

Acceso simple por clave y alta velocidad.

Columnar

Escala distribuida y grandes volúmenes.

Grafo

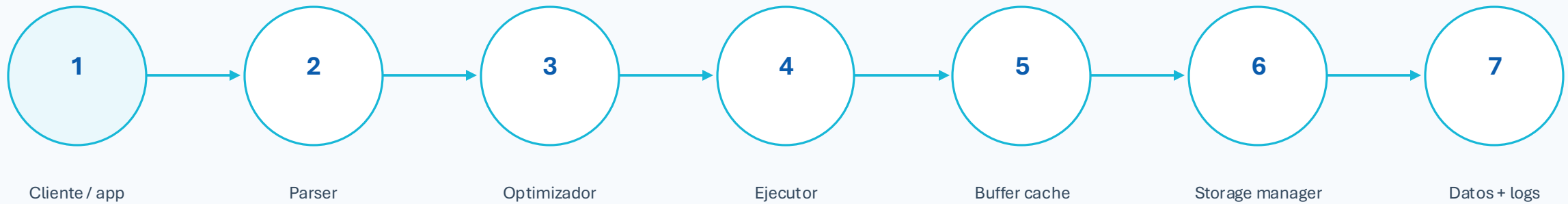
Nodos, aristas y relaciones complejas.

Otros ...

Series temporales, Motor de búsqueda, Orientado a objetos, Multimodelos

Arquitectura interna de un DBMS relacional

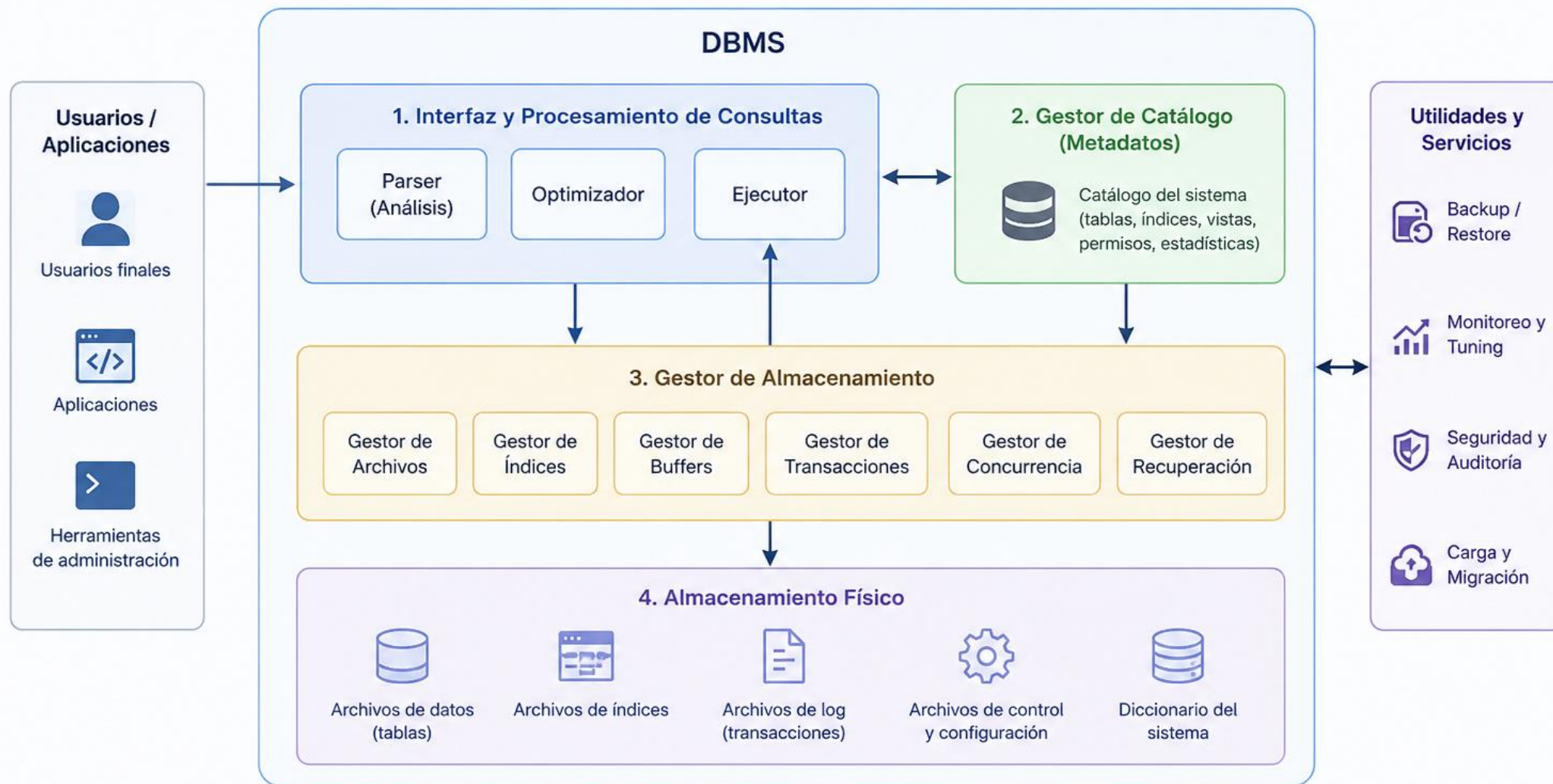
Del SQL al acceso físico



Cada motor tiene nombres y componentes propios, pero el patrón general se repite.

Arquitectura General de un DBMS (Modular)

Vista lógica de componentes y su interacción



Flujo de interacción

—> Flujo principal de operaciones

- - -> Flujo de consulta / referencia



Componentes desacoplados que colaboran para ofrecer servicios de datos confiables, eficientes y seguros.

Flujo simplificado de una consulta

Ejemplo: consultar el saldo de una cuenta

```
SELECT saldo FROM cuentas WHERE nro_cuenta = 12345;
```

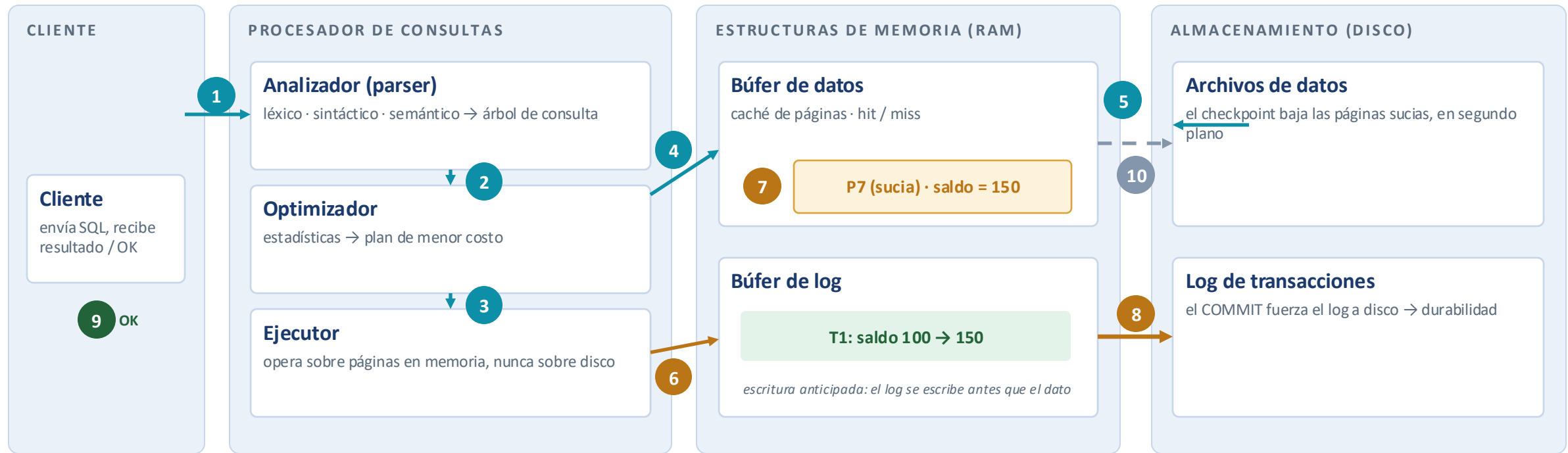
- El parser valida sintaxis y objetos.
- El catálogo/diccionario confirma columnas, tipos e índices.
- El optimizador elige un plan de acceso.
- El ejecutor lee desde memoria o solicita páginas al almacenamiento.
- El motor devuelve el resultado respetando aislamiento y permisos.

En en campus (sección de recursos generales) pueden encontrar un simulador explicado para visualizar dinámicamente las operaciones internas de un motor relacional generico.



Simulador de operaciones genérico de base de datos relacionales

El recorrido de una consulta SQL dentro del motor



● 1-5 lectura y plan (igual en SELECT) ● 6-8 escritura: log (6) antes que la página (7), COMMIT (8) ● 10 checkpoint (asíncrono)

Idea clave: el COMMIT persiste el log, no los datos. La página modificada queda “sucia” en RAM y baja al archivo recién con el checkpoint. Si el servidor cae, la recuperación rehace los cambios desde el log.

Procesamiento SQL y optimizador

La consulta no se ejecuta “tal como está escrita”

1

Parser

Comprueba sintaxis y transforma SQL en una representación interna.

2

Catálogo

Consulta metadatos: tablas, columnas, índices, estadísticas y permisos.

3

Optimizador

Estima costos y decide plan: índice, join, orden, filtros, paralelismo.

4

Ejecutor

Realiza el plan y coordina lectura, bloqueos/versiones y devolución.

Investigar planes de ejecución es una habilidad central del DBA y del desarrollador backend.

Memoria: el puente entre SQL y disco

La mayor parte del rendimiento pasa por la memoria



Buffer pool / cache

Mantiene páginas de datos e índices leídas recientemente.



Plan cache

Reutiliza planes o compilaciones de consultas frecuentes.



Sort / work memory

Usada para ordenamientos, joins, hash y operaciones intermedias.



Log buffer

Agrupar cambios antes de escribirlos en logs persistentes.

MySQL habla de InnoDB Buffer Pool; PostgreSQL de Shared Buffers; SQL Server de Buffer Pool; Oracle de SGA/Buffer Cache.

Almacenamiento físico

El motor traduce objetos lógicos en estructuras persistentes



Datos

Páginas, bloques, segmentos, heap, tablespaces o datafiles.



Índices

B-Tree, hash, columnstore, full-text, spatial, entre otros.



Logs

Registro transaccional para durabilidad y recuperación.



Temporales

Espacios para sort, hash, operaciones intermedias y objetos temporales.

La arquitectura física importa para rendimiento, backup, recuperación, crecimiento y costos.

Transacciones y propiedades ACID

La arquitectura protege operaciones críticas

A **Atomicidad**

Todo o nada.

C **Consistencia**

Respetar reglas e integridad.

I **Aislamiento**

Controla concurrencia.

D **Durabilidad**

Lo confirmado sobrevive fallas.

Los logs transaccionales, bloqueos, control de concurrencia y checkpoints son mecanismos concretos para sostener ACID.

Logs transaccionales

Diferentes nombres, una función esencial

MySQL/InnoDB

Redo log + undo log

PostgreSQL

WAL: Write-Ahead Log

SQL Server

Transaction log

Oracle

Redo log + undo

Patrón común: antes o junto con la modificación de datos, el motor registra información suficiente para rehacer o deshacer cambios.

Metadatos y diccionario de datos

El motor también almacena información sobre la información

Qué contiene

Tablas, columnas, tipos, restricciones, índices, vistas, estadísticas, usuarios, permisos, dependencias y configuración relevante.

Para qué se usa

Validar consultas, aplicar seguridad, estimar costos, construir planes de ejecución, documentar objetos y administrar el ciclo de vida.

Sin metadatos no hay optimizador, seguridad ni administración profesional.

El catálogo de metadatos en los 4 motores más usados

Dónde guarda cada motor la información sobre sus propios objetos, y cómo consultarla

Oracle Database

Catálogo: Diccionario de datos: vistas USER_* / ALL_* / DBA_* según alcance

```
SELECT table_name FROM user_tables;  
SELECT column_name, data_type FROM user_tab_columns WHERE table_name =  
'CUENTAS';
```

Visualización: SQL Developer (árbol de objetos, pestaña Columns/Details)

MySQL

Catálogo: INFORMATION_SCHEMA (estándar ANSI) + performance_schema

```
SHOW TABLES; DESCRIBE cuentas;  
SELECT table_name, engine FROM information_schema.tables WHERE  
table_schema = 'banco';
```

Visualización: MySQL Workbench (panel Schemas, Table Inspector)

Microsoft SQL Server

Catálogo: Vistas de catálogo sys.* + INFORMATION_SCHEMA + procedimientos sp_*

```
SELECT name, create_date FROM sys.tables;  
EXEC sp_help 'cuentas';
```

Visualización: SQL Server Management Studio (Object Explorer)

PostgreSQL

Catálogo: Catálogo pg_catalog (pg_class, pg_attribute) + information_schema

```
\dt \d cuentas (metacomandos de psql)  
SELECT relname FROM pg_class WHERE relkind = 'r';
```

Visualización: pgAdmin (Browser de objetos, pestaña Properties)

Idea clave: los metadatos son datos: cada motor los guarda en tablas y se consultan con SQL. **INFORMATION_SCHEMA** es el estándar ANSI que MySQL, SQL Server y PostgreSQL comparten; Oracle usa su propio diccionario.

Instancia, base de datos y esquema

El mismo término puede cambiar de significado según el motor

Concepto	Idea general	Prestar atención
Instancia	Proceso/servicio que administra recursos y conexiones.	No siempre equivale a una base de datos.
Base de datos	Contenedor lógico de objetos y datos.	En algunos motores se comporta como catálogo.
Esquema	Namespace o propietario lógico de objetos.	En Oracle suele vincularse al usuario; en SQL Server a ownership. En MySQL es equivalente a una BD
Catálogo	Metadatos del sistema y objetos disponibles.	La terminología no es idéntica entre plataformas.

Actividad clave: comparar términos antes de comparar comandos.

Comparación multiplataforma I

MySQL/InnoDB y PostgreSQL

MySQL / InnoDB

Instancia mysqld. Bases/esquemas como contenedores lógicos. InnoDB es el motor transaccional principal. Usa buffer pool, redo/undo, tablespaces e índices B-Tree. Muy usado en aplicaciones web y servicios transaccionales.

PostgreSQL

Cluster/instancia con varias bases. Esquemas como namespaces dentro de una base. Usa shared buffers, WAL, MVCC (Control de Concurrency Multiversión), catálogos del sistema y extensibilidad fuerte. Muy usado por robustez SQL y capacidades avanzadas.

Para pensar: ¿qué decisiones de arquitectura explican sus diferencias de administración y rendimiento?

Lo desarrollamos mas adelante en las actividades prácticas

Comparación multiplataforma II

SQL Server y Oracle Database

SQLS SQL Server

Instancia que contiene bases de datos. System catalogs, data files, log files y tempdb. Usa buffer pool, plan cache, transaction log y herramientas integradas de administración. Fuerte presencia empresarial.

ORA Oracle

Arquitectura con instancia, procesos, memoria y base de datos. Esquemas ligados a usuarios. Usa SGA/PGA, datafiles, control files, redo, undo y data dictionary. Muy usado en misión crítica.

Comparar motores implica entender nombres distintos para patrones parecidos.

Bases de datos en la nube

DBaaS cambia la operación, no elimina la arquitectura



Azure SQL / SQL Managed Instance

Servicio administrado sobre ecosistema SQL Server.



Amazon RDS / Aurora

Motores administrados y variantes optimizadas para cloud.



Cloud SQL

MySQL, PostgreSQL y SQL Server administrados en Google Cloud.

- El proveedor administra parte del backup, patching, disponibilidad y monitoreo.
- El equipo sigue definiendo modelo, consultas, índices, seguridad, costos y continuidad.

El DBA moderno

De administrar objetos a operar plataformas de datos



Operación

Instalación, configuración, monitoreo y continuidad.



Rendimiento

Planes, índices, estadísticas, memoria y carga.



Seguridad

Usuarios, roles, cifrado, auditoría y mínimos privilegios.



Automatización

Scripts, pipelines, observabilidad, IaC y documentación.



Costos

Dimensionamiento, licencias, cloud, almacenamiento y crecimiento.

La competencia clave es justificar decisiones técnicas según contexto organizacional.

Síntesis de la teoría

Patrones comunes que deben quedar claros

- Un DBMS separa vistas, modelo lógico y almacenamiento físico.
- Toda consulta atraviesa procesamiento, metadatos, optimización y ejecución.
- Memoria, logs, datos, índices y temporales sostienen rendimiento y recuperación.
- Instancia, base, esquema y catálogo cambian de significado según el motor.
- La comparación multiplataforma permite elegir tecnología con criterio.

Ahora pasamos de escuchar a investigar en forma individual o de a pares.

Seguimiento rápido en Campus Virtual

Criterios de evaluación en clase...

Criterio	Evidencia esperada	Nivel
Comprensión conceptual	Diferencia instancia, base, esquema, memoria, logs y metadatos.	Logrado / Parcial / No logrado
Comparación técnica	Contrasta al menos dos motores con fuentes verificables.	Logrado / Parcial / No logrado
Aplicación al caso	Justifica una decisión o mecanismo con lenguaje técnico propio.	Logrado / Parcial / No logrado
Uso responsable de IA	Declara uso, valida información y puede defender lo entregado.	Logrado / Parcial / No logrado

Cierre: cada uno comparte una diferencia relevante y una duda técnica para la próxima clase.

Actividad 1 · Matriz comparativa

45 minutos · investigación guiada · entrega en Campus Virtual

Consigna

Cada alumno investiga un motor asignado y completa una matriz comparativa: instancia, base de datos, esquema, memoria, archivos, logs, metadatos, herramienta administrativa y rasgo distintivo.

La premisa es NO usar IA sino ir a la página oficial del motor.

Entregable

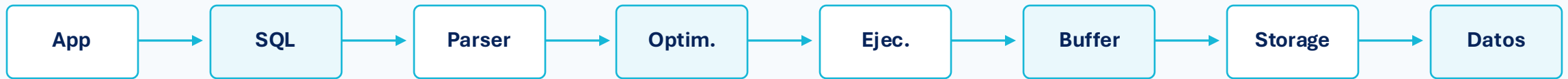
Tabla breve en Campus Virtual + una conclusión: “Lo más importante que diferencia a este motor es...”. Incluir fuente consultada.

Motores sugeridos: MySQL/InnoDB · PostgreSQL · SQL Server · Oracle · MariaDB · DB2

Resultado demostrable: matriz comparable de distintos motores relacionales.

Actividad 2 · Mapa visual de arquitectura

45 minutos · diagrama + explicación breve



Consigna

Construir un diagrama del flujo de una consulta y marcar memoria, metadatos, logs, datos e índices. Relacionarlo con el motor real investigado en la actividad anterior.

Entregable

Imagen o PDF del diagrama + explicación de 5 líneas. Puede realizarse en PowerPoint, draw.io, Excalidraw, Miro o Canva.

Resultado demostrable: diagrama defendible en 2 minutos.

IA como herramienta complementaria

Investigar arquitectura con criterio profesional



Uso permitido

Explicar conceptos, generar analogías, comparar componentes, sugerir comandos de exploración, revisar diagramas y mejorar documentación.



Uso no aceptable

Copiar respuestas sin validar, ocultar el uso, citar información inexistente, no poder defender el diagrama o confundir nombres entre motores.

Regla de oro: la IA acelera la búsqueda, pero la evidencia debe poder explicarse y verificarse.